

Diagnosing and Correcting a Volume Confound in Proxy-Label Supply Chain Stress Prediction: A Validation-at-Scale Case Study on the M5 Retail Dataset

Suhail Yunus¹

¹Independent Researcher

Abstract

Supervised models for supply chain risk frequently rely on proxy labels, since ground-truth outcomes such as stockouts are rarely recorded in public retail datasets. We construct one such proxy – an item-store “demand stress” label derived from the M5 Forecasting dataset [Makridakis et al., 2022] – and use it as a case study in the validation discipline that proxy-label modeling requires. We report three findings, each obtained by testing an assumption rather than asserting it. First, two chronological data-leakage defects (a threshold estimated over a series’ full history rather than its training period, and a rolling feature that included the current observation) were present in an early draft of the pipeline and were caught only by explicit regression tests, not by holdout performance, which looked plausible in both cases. Second, an external review identified that our stress threshold, computed by pooling sales across all ten stores per item, let high-volume stores exceed the threshold more often simply by selling more; we verified this directly (store-level stress rate correlated with store-level sales volume at Pearson $r = 0.85$) and corrected it by grouping the threshold on (item, store) rather than item alone, which dropped the correlation to $r = 0.034$ and reduced recall at every operating threshold – the honest result of removing a labeling artifact, not a regression. Third, we tested whether the resulting model’s predicted scores are genuine probabilities, using a three-way chronological split that never lets calibration-set data influence the final report. The model was confirmed severely miscalibrated at both a 100-item and a full 30,490-series scale (Brier score improving from 0.210 to 0.076, and from 0.227 to 0.105, respectively, under isotonic calibration), yet the calibrated and uncalibrated models’ cost-optimal operating points achieve nearly identical business outcomes, because monotonic calibration preserves rank order and the operating decision depends only on rank. We conclude that in proxy-label, class-imbalanced pipelines, an implicit grouping choice in the label definition is a substantive modeling decision that ordinary train/test evaluation does not surface, and that probability calibration and threshold selection are separable concerns: the first governs whether a score can be trusted as a probability, the second depends only on ranking. All code, tests, and artifacts are available in the accompanying repository.

1 Introduction

Predicting operational risk in a supply chain – which items, at which locations, are likely to face unusual demand pressure – is a problem most public retail datasets cannot answer directly, because they do not record the outcome that actually matters: whether a stockout, a supplier delay, or an unfulfilled order occurred. Practitioners building such models are therefore routinely forced into *proxy-label* modeling: defining a measurable stand-in for the unobserved outcome (here, unusually high demand relative to a series’ own history) and treating it as ground truth for supervised learning.

Proxy labels carry a risk that ordinary machine learning validation practice does not automatically catch. A model can achieve strong, stable holdout performance while the label itself encodes an artifact of how it was constructed rather than the phenomenon it is meant to stand in for. Standard train/test splits, cross-validation, and even chronological holdout evaluation – the discipline this paper otherwise argues for – are silent on whether the *label definition* itself is sound, because every one of those procedures evaluates a model’s ability to predict the label as given, not whether the label means what it is assumed to mean.

This paper documents a case where exactly that happened, how it was found, and what changed once it was fixed. Using the M5 Forecasting – Accuracy dataset [Makridakis et al., 2022], we define a stress label as sales exceeding an item’s own historical 90th percentile, intended to proxy for unusual demand pressure. An early version of this label, and the pipeline built on it, held up under every check we initially ran: a chronological holdout, a threshold estimated only from training-period history, and stable performance across a 3.3x change in sample size. It nonetheless contained a labeling artifact that inflated the model’s apparent ability to detect risk, discovered only when an external reviewer questioned an assumption already written down as an open limitation in our own documentation but never tested.

We make three contributions, each in the form of a tested claim rather than an assumed one:

1. We document two chronological data-leakage defects found during initial development – a stress threshold estimated over each series’ complete history rather than its training-period history alone, and a rolling-window feature that included the current day’s own value – and show that standard holdout evaluation did not surface either one; both were caught only by explicit regression tests asserting temporal independence (Section 4).
2. We document a store-volume confound in the label definition itself: pooling sales across all ten stores per item when computing the stress threshold let high-volume stores exceed it more often simply by selling more, independent of genuine demand anomalies. We verify this quantitatively (Pearson $r = 0.85$ between store-level stress rate and store-level sales volume), correct it, and re-validate the corrected pipeline at three scales – 1,000, 5,000, and the complete 30,490-series catalog (Section 5, Section 6).
3. We test, rather than assume, whether the model’s predicted scores are calibrated probabilities, using a three-way chronological split (train / calibration / held-out final evaluation) that keeps the final report clean of any data used to fit the calibrator. We show the model is severely overconfident at both a small and a full-catalog scale, and that calibration corrects this without materially changing the cost-optimal operating decision – because that decision depends on the rank ordering of scores, which monotonic calibration methods do not alter (Section 7).

We take no position on whether gradient-boosted trees are the right model family for this problem in general; Section 8 reports a controlled comparison against two commonly-used alternatives, and finds that the original choice was already well-supported rather than arbitrary. The contribution of this paper is not a novel algorithm but a documented validation methodology for proxy-label, class-imbalanced, cost-sensitive prediction problems, illustrated on a case where every step of that methodology was necessary to catch a real error.

2 Related Work

Benchmark dataset. The M5 Forecasting competition [Makridakis et al., 2022] evaluated point-forecast accuracy on 42,840 hierarchical unit-sales series from Walmart; we use its constituent

sales, calendar, and pricing files but repurpose the task from demand forecasting to binary stress classification, a proxy-label reframing not evaluated in the original competition.

Data leakage. Kaufman et al. [2012] formalize leakage in predictive modeling as the introduction of information about the target that would not legitimately be available at prediction time, and argue that a strict *learn-predict separation* is the primary defense. The two leakage defects we report (Section 4) are concrete instances of exactly this failure mode in a time-series setting, where the temporal ordering of the data is the specific axis along which separation must be enforced.

Demand classification. Syntetos et al. [2005] propose classifying demand series by average inter-demand interval and the squared coefficient of variation of demand sizes, yielding the smooth/erratic/intermittent/lumpy taxonomy we use to design a stratified sample (Section 6) that deliberately oversamples the harder, more business-relevant intermittent and lumpy segments rather than sampling uniformly at random.

Gradient boosting. We use XGBoost [Chen and Guestrin, 2016] as the primary model and compare it against LightGBM [Ke et al., 2017] and CatBoost [Prokhorenkova et al., 2018] under matched hyperparameters and identical class-imbalance handling (Section 8).

Probability calibration. Niculescu-Mizil and Caruana [2005] show that boosted trees systematically distort predicted probabilities away from 0 and 1 in a characteristic sigmoid-shaped pattern, and evaluate Platt scaling [Platt, 1999] and isotonic regression [Zadrozny and Elkan, 2002] as corrections. Our finding (Section 7) that a boosted-tree model trained with class-imbalance reweighting is severely miscalibrated is consistent with this literature; our contribution is showing, on a real cost-sensitive decision problem, that the correction changes what a score *means* far more than it changes which decision the score implies.

3 Dataset and Proxy Target Definition

M5 provides daily unit sales for 3,049 items across 10 stores in 3 U.S. states (30,490 item-store series in total), spanning 1,913 days, together with a calendar file (weekday, month, year, event flags, SNAP eligibility) and a weekly item-store price file.

No inventory, stockout, or supplier-delay signal exists anywhere in M5. We therefore define a proxy target:

$$\text{stress_event}_{i,s,t} = \mathbb{I}[\text{sales}_{i,s,t} > Q_{0.90}(\text{sales}_{i,s,\cdot} \mid t' \leq t_{\text{split}})] \quad (1)$$

for item i , store s , day t , where $Q_{0.90}$ is the 90th percentile of that series’ sales, estimated *only* from days on or before the chronological train/holdout split day t_{split} , then applied as a fixed threshold across the entire series including the holdout period. This threshold-estimation rule is itself a leakage safeguard: estimating $Q_{0.90}$ over a series’ complete history (including holdout-period days) would let information about future sales inform a label applied to training-period rows, and no such future-informed threshold could be computed at real prediction time in production. Section 5 describes a defect in the *grouping* used in this definition, which is distinct from and orthogonal to this temporal safeguard.

The 80/20 chronological train/holdout split day is derived once from the full 1,913-day range and is scale-invariant: because every series shares an identical day range, the split day computed on a 100-item sample is identical to the split day computed on the full 30,490-series catalog, which we confirmed empirically holds exactly (day 1,531) across every sample size tested.

4 Leakage-Safe Pipeline Design

Two chronological leakage defects were identified and fixed during initial development, before any of the findings reported in the remainder of this paper. Both are documented here because neither was surfaced by holdout performance, which looked plausible in both the leaking and corrected versions – they were caught only by writing an explicit test that asserted the *mechanism* of leakage-safety, not merely that overall metrics looked reasonable.

Threshold estimation window. An early implementation estimated the 90th-percentile stress threshold (Section 3) over each series’ complete history, including holdout-period days. This is leakage under Kaufman et al. [2012]’s formulation: the label applied to a training-period row was informed, in part, by sales that occur after that row in time. The fix constrains threshold estimation to training-period days only, as stated in Equation 1.

Rolling-feature window. The 7-day rolling mean and standard deviation features were initially computed as a trailing window inclusive of the current day, so that a demand spike would appear in its own rolling-average feature. The fix shifts the window by one day before applying it (`sales.shift(1).rolling(7)` in the pandas implementation; `ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING` in the equivalent SQL), so that day t ’s features never include day t ’s own sales value.

Both defects were fixed prior to any of the results in Section 5 onward, and both are now guarded by regression tests that construct a synthetic series with a known, hand-computable spike and assert the exact numeric value of the resulting feature – a test strong enough to fail immediately if either defect were reintroduced.

5 The Store-Volume Confound

5.1 Discovery

The stress-target definition in Section 3 groups the 90th-percentile threshold calculation by item alone, pooling sales across all 10 stores that carry that item. This was flagged as an open, untested concern in our own documentation (“the observed store-level feature importance may partly reflect the label definition rather than genuine demand behaviour ... this has not been evaluated”) for an extended period before an external reviewer (a large language model used for code review, Google Gemini) identified the same concern independently and prompted us to actually test it.

The mechanism is direct: pooling sales across stores means a single threshold is applied to every store carrying that item, regardless of each store’s individual sales volume. A store that sells more units of an item on average will exceed that pooled threshold more often, independent of whether its demand is genuinely more volatile relative to its own baseline.

5.2 Verification

We computed, for each of the 10 stores, the empirical stress-event rate on the holdout period under the original item-only grouping, and the store’s average daily sales volume across the full dataset. Table 1 reports the result.

A correlation of $r = 0.85$ between a store’s stress-event rate and its raw sales volume is not consistent with the label measuring demand *anomalies*; it is consistent with the label partly measuring which stores sell more units in general.

Table 1: Store-level stress rate before and after correcting the label grouping (30,490-series catalog).

	Item-only grouping (flawed)	Item+store grouping (corrected)
Stress-rate range across 10 stores	7.7%–19.9%	10.3%–16.4%
Pearson r (rate vs. store sales volume)	0.85	0.034

5.3 Correction and consequence

We corrected the grouping to (item, store) – i.e., each item-store series gets its own 90th-percentile threshold, estimated from that series’ own training-period history alone, rather than pooling across the 10 stores that carry the item. This is a strictly more conservative definition: it removes exactly the volume-driven component of the label without introducing any new information.

The corrected grouping dropped the store-rate/volume correlation to $r = 0.034$ (Table 1) and, at every scale tested, reduced recall at every operating threshold relative to the flawed definition (Section 6). We regard this as the correct outcome of removing a labeling artifact, not a regression: a portion of the original model’s apparent recall was attributable to learning “which item-store combinations sell a large volume,” which is not the phenomenon the label was designed to capture.

5.4 A second defect surfaced by the fix

Re-validating the corrected label at full catalog scale required processing the 58.3-million-row long-format dataset in per-store chunks to fit within available memory (Section 6), since the full dataset does not fit in memory on typical development hardware. This chunking approach silently introduced a second defect: one-hot encoding the `store_id` and `state_id` location features against *only the categories present in the current chunk* produces zero columns when a chunk contains a single store, since one-hot encoding a column with exactly one observed category and `drop_first=True` has nothing left to encode. This failure is silent – no exception is raised, no warning is emitted – and a joint model trained across all chunks would have been trained with no location information whatsoever, despite location features appearing to be present in the pipeline’s code path. We caught this by asserting that every chunk’s output schema was identical before trusting any downstream result, and fixed it by encoding against a fixed, externally-supplied list of all 10 stores and 3 states, so that every chunk produces the same, stackable set of columns regardless of which single store it contains. This defect is unrelated to the store-volume confound but was found in the course of investigating it, and is reported here because it exemplifies the same general risk: a validated-looking pipeline stage that is silently wrong in a way ordinary execution does not reveal.

6 Validation at Scale

6.1 Stratified sampling design

Beyond a 100-item development sample (equivalent to 1,000 series, since every item spans all 10 stores), we constructed a stratified 5,000-series sample by drawing 500 items stratified by department and by demand-pattern class, using the Syntetos et al. [2005] taxonomy computed from each item’s average inter-demand interval and the squared coefficient of variation of its non-zero demand sizes. Intermittent and lumpy items – the more business-relevant and empirically harder segment – were oversampled by a factor of 1.5 relative to their natural catalog share (44% → 53% intermittent representation in the sample). Because every item spans all 10 stores, sampling 500 items yields

exactly 5,000 series with no store-representation bias.

6.2 Full-catalog processing

The complete catalog (30,490 series, 58.3 million long-format rows) does not fit in memory on the reference development hardware used for this work (a 3.9 GB, single-core container). We processed it store-by-store (10 chunks of approximately 5.8 million rows each), checkpointing each chunk to disk before processing the next, so that an interruption loses at most one chunk of work rather than the entire run. For the business-impact results reported in this section, the model was trained on a 20% row-subsample of the training period for computational tractability; the holdout evaluation itself was never subsampled, since the final business-impact figures are the numbers that matter most and should not be the ones approximated.

6.3 Results across scale and correction

Table 2 reports the shipped-threshold (0.80, in raw uncalibrated score units) and cost-optimal operating point at each combination of scale and label-grouping correctness we tested, under the cost assumptions of \$20 per missed stress event, \$12 per false alarm, and \$10 per correctly-flagged event requiring review.

Table 2: Business-impact outcome across scale and label correction. Threshold values are raw (uncalibrated) scores.

Scale	Label grouping	Net at threshold 0.80	Cost-optimal (threshold, net)
1,000 series	Item-only (flawed)	−\$50,414	(0.95, +\$26)
5,000 series (3-seed avg.)	Item-only (flawed)	−\$53,158	(~0.88, +\$57,877)
5,000 series	Item+store (corrected)	−\$9,078	(0.89, +\$27,584)
30,490 series (full)	Item+store (corrected)	−\$209,794	(0.90, +\$47,148)

Two findings hold across every row of Table 2. First, the shipped threshold of 0.80 consistently loses money relative to taking no action at all, at every scale and under both the flawed and corrected label definitions; this is a stable, scale-independent conclusion, not an artifact of small-sample noise. Second, a cost-optimal operating threshold exists near the top of the raw score range, and the magnitude of its benefit is not stable across sample draws: repeating the 5,000-series sampling procedure with 3 different random seeds produced a cost-optimal net ranging from +\$37,850 to +\$85,500 – a $2.3\times$ spread from seed choice alone. We report this instability explicitly because a single-seed result at this sample size would have supported a specific dollar claim that does not replicate.

7 Probability Calibration

7.1 Motivation and methodology

Every operating threshold reported in Section 6 is a raw XGBoost score. `scale_pos.weight`, used throughout this work to counteract severe class imbalance (approximately 12:1 negative-to-positive in the training data), is known to distort a boosted-tree model’s predicted probabilities upward for the positive class [Niculescu-Mizil and Caruana, 2005]; a raw score of 0.80 is therefore not a

claim that an event is 80% likely, and treating it as one – for instance, when communicating alert confidence to a human operator – is unsupported until tested.

We test this directly rather than continue to assert it as an untested limitation, using a three-way chronological split that separates model training, calibrator fitting, and final reporting into three disjoint, chronologically ordered periods:

$$\text{train} (\text{day} \leq t_{\text{split}}) \rightarrow \text{calibration} (t_{\text{split}} < \text{day} \leq t_{\text{calib}}) \rightarrow \text{final_eval} (\text{day} > t_{\text{calib}}) \quad (2)$$

This three-way split matters because reusing part of the existing holdout period to fit a calibrator would mean that portion is no longer a genuinely held-out set for the final report; splitting the holdout period itself into a calibration segment and a strictly later final-evaluation segment keeps every “before vs. after” number in this section honestly held out. We fit both Platt/sigmoid [Platt, 1999] and isotonic [Zadrozny and Elkan, 2002] calibrators via `FrozenEstimator` (scikit-learn ≥ 1.6 , the modern replacement for the deprecated `cv="prefit"` argument) wrapping the already-trained base model, so that calibrator fitting never refits or otherwise touches the base model.

7.2 Results

Table 3 reports Brier scores at both a 100-item and the full 30,490-series scale. Figure 1 and Figure 2 show the corresponding reliability diagrams.

Table 3: Brier score before and after isotonic calibration (lower is better; 0 = perfect, 0.25 = no better than the base rate).

Scale	Raw	Calibrated (isotonic)
100-item	0.210	0.076
Full catalog (30,490 series)	0.227	0.105

At the full-catalog scale, sweeping the operating threshold over both the raw and calibrated scores gives a cost-optimal net of +\$30,254 (raw, at threshold 0.89) versus +\$30,138 (calibrated, at threshold 0.51) – a difference of \$116, well within the noise already documented in Section 6.

7.3 Why calibration barely changes the business decision

This near-equality is not coincidental. Isotonic and Platt/sigmoid calibration are both *monotonic transformations*: they can relabel a score, but by construction cannot change the rank order of the observations being scored. A cost-optimal threshold search finds the best-ranked subset of observations to flag, given the specified costs; since the ranking is unchanged by calibration, the optimal *decision* – which observations get flagged – is nearly unchanged as well, even though the numerical threshold that produces that decision looks very different before and after calibration (0.80 raw vs. 0.28 calibrated, at the 100-item scale, for a matched alert volume; see Section 7.4).

This separates two questions that are easy to conflate when reading a single threshold number off a model: *which threshold produces the best decision* depends only on ranking, and calibration does not change it; *what a given score means as a stated probability* depends on calibration entirely, and matters whenever a score is communicated to a human, combined with another model’s probability estimate, or used in any calculation that assumes it is a genuine likelihood rather than a ranking value.

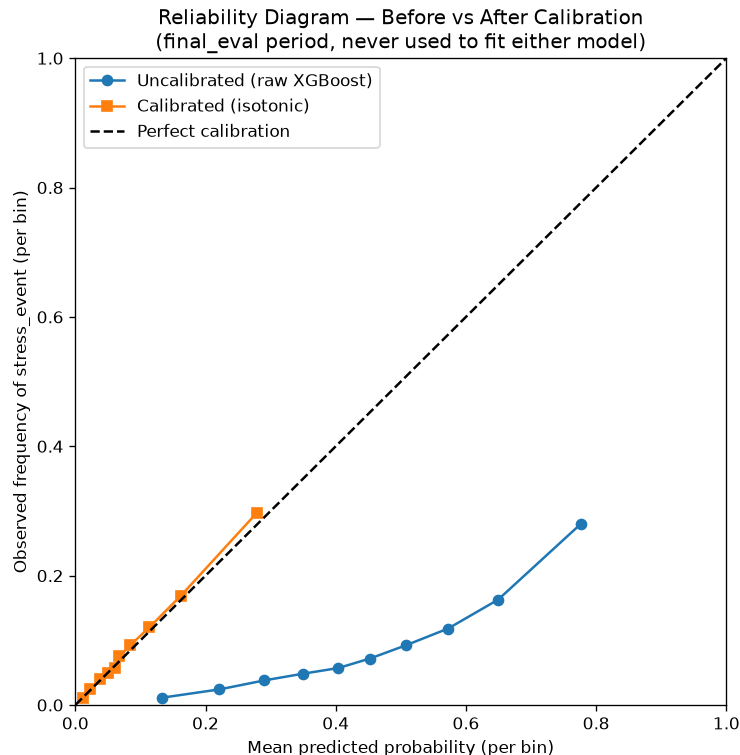


Figure 1: Reliability diagram, 100-item scale, final-evaluation period only. A raw score of approximately 0.8 corresponds to an observed positive rate of approximately 0.28; the calibrated curve tracks the diagonal.

7.4 Deployment

We deployed the calibrated model as the production artifact rather than leaving this as a documented-but-unshipped finding. Deploying required identifying an operating threshold in calibrated-probability space that preserves the same alert volume the original raw threshold of 0.80 was chosen to produce – since that raw threshold was itself selected for a workload reason (a reviewable daily alert volume for a single analyst) rather than because 0.80 was a materially meaningful probability. We searched the empirical calibrated-score distribution for the threshold yielding the closest alert count to the original, since isotonic regression’s tied “plateau” values mean an exact match is not always achievable at a single cutoff; the resulting threshold (0.2801) reproduces the original alert volume within 3.6%. All existing regression tests pass against the deployed calibrated artifact; two tests that had hardcoded an assumption that the deployed model type would always be `XGBClassifier` were revised to assert the actual behavioral contract the serving code depends on (`predict_proba`) instead of one specific implementation’s class.

8 Model Comparison

XGBoost was selected at the start of this project without being tested against alternatives. We report a controlled comparison against LightGBM [Ke et al., 2017] and CatBoost [Prokhorenkova et al., 2018], trained on identical data, identical features, an identical chronological train/holdout split, and identical class-imbalance handling (`scale_pos_weight` computed identically for all three

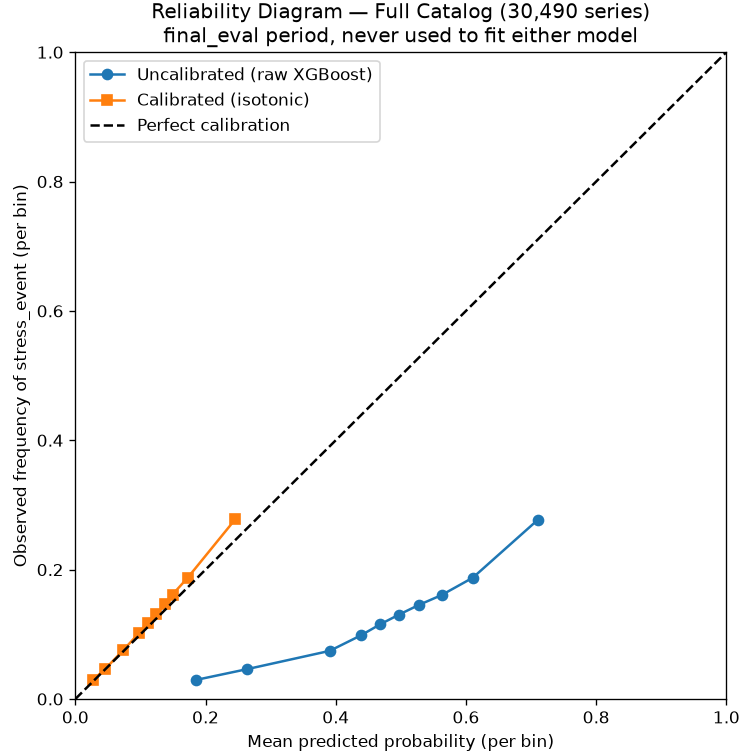


Figure 2: Reliability diagram, full 30,490-series catalog, final-evaluation period only. The same overconfidence pattern is present at full scale.

libraries’ equivalent parameter, at matched ~ 200 -tree, depth-8 model complexity, with no additional tuning given to any one model).

Table 4: Model comparison, 100-item scale, corrected label definition.

Model	Average precision	Brier score (raw)	Cost-optimal net
XGBoost (shipped)	0.236	0.210	\$0
LightGBM	0.230	0.214	\$0
CatBoost	0.220	0.214	\$16

XGBoost achieves the highest average precision, the threshold-independent metric we consider most meaningful for this comparison. The cost-optimal net differences (\$0 / \$0 / \$16) are not distinguishable from noise at this sample size, and are consistent with the near-breakeven business case already established at the 100-item scale in Section 6. We conclude that the original model choice was already reasonably well-supported rather than arbitrary; this is a case where testing an assumption confirmed it, which we report plainly rather than treat as a null result unworthy of inclusion.

9 Discussion

The unifying theme across Sections 4–8 is that each finding required deliberately testing a specific, checkable claim, rather than trusting that a pipeline producing stable, plausible-looking holdout

metrics was therefore correct. Standard chronological holdout evaluation – which we used throughout, and which is necessary – was insufficient by itself in every case reported here: it did not surface either leakage defect, it did not surface the store-volume confound (the confounded model’s holdout performance was internally consistent and plausible-looking), and it does not by itself indicate whether a model’s scores are calibrated probabilities.

We highlight two more specific implications for practitioners building similar proxy-label, class-imbalanced, cost-sensitive systems. First, a *grouping choice* embedded in a label definition – in this case, whether a percentile threshold pools across a hierarchy level (store) that is not the unit of prediction – is a substantive modeling decision with the same potential for confounding as an explicit predictive feature, yet it is easy to treat as an implementation detail rather than a decision requiring its own validation. We found it only because an external reviewer questioned an assumption we had ourselves flagged as untested; we had no automated safeguard against it until after the fact. Second, probability calibration and threshold selection are separable engineering concerns that address different failure modes and should not be conflated: a model’s ranking quality, and therefore its cost-optimal decision, can be entirely sound while its scores are simultaneously unusable as stated probabilities, and fixing the second does not imply anything changes about the first.

10 Limitations

The stress label remains a proxy, not a verified stockout or supply-disruption outcome; M5 contains no inventory or fulfillment data against which to validate it directly. The cost assumptions used throughout Sections 6–8 (\$20 / \$12 / \$10 for missed events, false alarms, and reviewed true positives respectively) are stated, illustrative assumptions rather than values derived from a real operational cost study, and the business-impact conclusions in this paper should be read as conditional on those assumptions, not as a claim about the dollar magnitude of value in any specific real deployment. The full-catalog results in Sections 6 and 7 were obtained using different exact experimental pipelines developed at different points in this investigation (a store-chunked processing pipeline for the business-impact results in Section 6, and a separate three-way-split pipeline built specifically for the calibration check in Section 7), both applying the same corrected label definition and cost assumptions but not sharing a single unified codebase run; we report this plainly rather than imply a uniformity across these results that the underlying experimental history does not have. Finally, the recall achieved at any operating threshold that also yields a positive cost-optimal outcome is low in absolute terms (Section 6); this model is more appropriately characterized as a prioritization aid for a human review queue than an autonomous detection system, a characterization we maintain has not changed with any of the corrections reported in this paper.

11 Conclusion

We presented a validation-at-scale case study on a proxy-label supply chain stress classifier, in which two data-leakage defects, a store-volume confound in the label definition, and a severe probability miscalibration were each identified only by explicitly testing a specific claim rather than trusting plausible-looking aggregate performance. The store-volume confound, in particular, illustrates a failure mode specific to proxy-label modeling that ordinary train/test evaluation does not surface: a grouping choice in how a label is computed can encode a genuine confound while producing a model that looks, by every standard evaluation metric, entirely reasonable. We further show that calibrating a class-imbalance-reweighted model’s probabilities is necessary for the scores to be

trusted as stated likelihoods, but is close to irrelevant to the cost-optimal decision the model is used to make, since that decision depends only on score ranking, which calibration does not alter. We make all code, tests, and the artifacts behind every reported number available so that each of these findings can be independently reproduced or challenged.

References

- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):Article 15.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154.
- Makridakis, S., Spiliotis, E., and Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4):1346–1364.
- Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 625–632.
- Platt, J. (1999). Probabilistic outputs for support vector machines and comparison to regularized likelihood methods. *Advances in Large Margin Classifiers*, pages 61–74.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems 31*, pages 6638–6648.
- Syntetos, A. A., Boylan, J. E., and Croston, J. D. (2005). On the categorization of demand patterns. *Journal of the Operational Research Society*, 56(5):495–503.
- Zadrozny, B. and Elkan, C. (2002). Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 694–699.